React + TypeScript · 10-Day Training Plan

Day 4 — State Management Basics

Your simple, friendly guide for today

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Make components interactive: give them memory (state) with useState so the screen updates when the user clicks or types.

How to use this: Read **Section 1 (Learning Plan)** for each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make it click. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan

Welcome to Day 4! On Day 3 your components could **show** data (props, lists, composition), but the screen never changed after it loaded. Today the UI comes alive.

State = a component's own memory. It is data that can change over time (like text in a search box), and when it changes, React automatically redraws the screen for you.

> **This is the big one for you.** React state is the concept that most resembles **KnockoutJS observables**. In Knockout you made a value "live" with `ko.observable( ... )`, and the UI updated when it changed. React's `useState` does the same job — you'll feel right at home.

We'll build toward one small app the whole day: an **editable employee list with a search box and a department filter**. The state lives in a parent, gets passed down to children, and the children raise changes back up through callback props. (This is used only as our running example — there's no task to complete here.)

Assumed setup for every snippet below (so they compile under strict React + TypeScript):

```
import { useState } from "react";
import React from "react"; // only in the snippet that uses React.ChangeEvent

// Same Employee shape you built on Day 3:
type Employee = {
  id: number;
  firstName: string;
  lastName: string;
  department: string;
  salary: number;
  isActive: boolean;
  joinDate: string;
};
```

Part A — The Core Topics

1. `useState` — giving a component memory

`useState` is a **hook** (a special function React gives you, always starting with `use`). You call it with the starting value, and it hands you back a pair: the **current value** and a **setter function** to change it. When you call the setter, React re-renders the component with the new value. Never change the value directly — always go through the setter.

```
function Counter() {
  const [count, setCount] = useState(0); // count starts at 0

  return <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>;
}
```

- **Coming from KnockoutJS / C#:** Like `this.count = ko.observable(0)` — but you read it as a plain variable (`count`) and change it with `setCount` , and React re-renders for you (no manual binding refresh).
- **You'll be able to:** give a component its own memory that survives between re-renders.

2. Handling events (`onClick` / `onChange`)

You react to the user by attaching a function to an event prop right inside the JSX. `onClick` runs when something is clicked; `onChange` runs on every keystroke in an input. React passes the handler an **event object** (`e`), and TypeScript types it for you when you write the handler inline. Usually the handler just reads a value and calls a state setter.

```
function Greeter() {
  const [name, setName] = useState("");

  return (
    <div>
      {/* onChange fires on every keystroke; onClick fires on click */}
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <button onClick={() => alert("Hi " + name)}>Greet</button>
    </div>
  );
}
```

- **Coming from KnockoutJS / C#:** Like KO's `click:` and `event:` bindings, or a C# `Button_Click` handler — except the handler is wired straight into the markup, not in a separate code-behind file.
- **You'll be able to:** respond to clicks and typing and turn them into state changes.

3. Conditional rendering driven by state

To show or hide part of the UI, you use plain JavaScript inside `{ }` : the ternary `condition ? A : B` to pick between two things, or `condition && <Thing/>` to show something only when the condition is true. Because the condition reads a **state** value, the UI changes the moment that state changes.

```
function EmployeePanel() {
  const [showInactive, setShowInactive] = useState(false);

  return (
    <div>
      <button onClick={() => setShowInactive(!showInactive)}>
        {showInactive ? "Hide inactive" : "Show inactive"}
      </button>
      {showInactive && <p>Now showing inactive employees too.</p>}
    </div>
  );
}
```

- **Coming from KnockoutJS / C#:** Like KO's `if:` and `visible:` bindings (or a Razor `@if`) — but here it's just normal JavaScript expressions inside `{ }`.
- **You'll be able to:** show, hide, or swap parts of the screen based on state.

4. Dynamic lists + keys driven by state

To render many items, keep them in a state **array** and use `.map()` to turn each item into JSX. Give every item a `key` — a stable, unique id that helps React track which item is which when the list changes. Use a real id (like `employee.id`), never the array index. When the state array changes, the list re-renders on its own.

```
function EmployeeNames() {
  const [employees] = useState<Employee[]>([
    { id: 1, firstName: "Asha", lastName: "Rao", department: "IT", salary: 90000, isActive:
true, joinDate: "2022-04-01" },
    { id: 2, firstName: "Ben", lastName: "Cole", department: "HR", salary: 70000, isActive:
true, joinDate: "2023-01-15" },
  ]);

  return (
    <ul>
      {employees.map((e) => (
        <li key={e.id}>{e.firstName} {e.lastName}</li> // key = stable id, not the array
index
      ))}
    </ul>
  );
}
```

- **Coming from KnockoutJS / C#:** Like KO's `foreach:` binding over an `observableArray` — but you write the loop yourself with `.map()`, and the `key` is your job.
- **You'll be able to:** render a list from a state array that redraws itself whenever the array changes.

5. Lifting state up

When two components need the same data, you don't duplicate the state in both. You move ("lift") it up into their nearest **parent**, then pass the value **down** as a prop. To let a child change it, you pass a **callback function** down too; the child calls it, and the parent updates its own state. This is the pattern behind our search + filter app: the parent owns everything, the inputs just report changes.

```
type SearchInputProps = {
  value: string;
  onSearchChange: (next: string) => void; // callback the parent gives us
};

function SearchInput({ value, onSearchChange }: SearchInputProps) {
  return <input value={value} onChange={(e) => onSearchChange(e.target.value)} />;
}

function SearchParent() {
  const [search, setSearch] = useState(""); // state lives HERE, in the parent

  return (
    <div>
      <SearchInput value={search} onSearchChange={setSearch} />
      <p>Searching for: {search}</p>
    </div>
  );
}
```

- **Coming from KnockoutJS / C#:** Like putting an observable on a parent view model and passing both the value and a setter down to child components — data flows down, events bubble up.
- **You'll be able to:** let separate children share and update the same piece of data through their parent.

6. Controlled inputs

A **controlled input** is a form field whose value is driven by state. You set two things: `value={state}` (state decides what's shown) and `onChange` (typing updates the state). React is one-way: state → input, and onChange → state. This is exactly what a search box or a department dropdown needs.

```
function DepartmentFilter() {
  const [department, setDepartment] = useState("All");

  return (
    <select value={department} onChange={(e) => setDepartment(e.target.value)}>
      <option value="All">All</option>
      <option value="IT">IT</option>
      <option value="HR">HR</option>
    </select>
  );
}
```

- **Coming from KnockoutJS / C#:** Like KO's two-way `value:` binding — except React splits it into two explicit halves you control (`value` + `onChange`).
- **You'll be able to:** build search boxes and dropdowns whose value always matches your state.
- **Heads-up:** typing values into fields is in scope today (we need it for search/filter). **Deep form handling and validation is Day 6**, and **loading data with `useEffect` is Day 5** — we're not doing those yet.

Putting it together (our running example)

Here's the target, using everything above. The parent **owns** `search` and `department` state, **derives** the filtered list during render, **passes** state down, and the child **raises** changes through callbacks. (Example only — nothing to submit.)

```
type ControlsProps = {
  search: string;
  department: string;
  onSearchChange: (next: string) => void;
  onDepartmentChange: (next: string) => void;
};

function EmployeeControls({ search, department, onSearchChange, onDepartmentChange }:
ControlsProps) {
  return (
    <div>
      <input
        placeholder="Search name..."
        value={search}
        onChange={(e) => onSearchChange(e.target.value)}
      />
      <select value={department} onChange={(e) => onDepartmentChange(e.target.value)}>
        <option value="All">All</option>
        <option value="IT">IT</option>
        <option value="HR">HR</option>
      </select>
    </div>
  );
}

function EmployeeDirectory({ employees }: { employees: Employee[] }) {
  // Parent OWNS the state...
  const [search, setSearch] = useState("");
  const [department, setDepartment] = useState("All");

  //  ...derives the filtered list during render (no extra state)...
  const visible = employees.filter((e) => {
    const matchesName = (e.firstName + " " + e.lastName)
      .toLowerCase()
      .includes(search.toLowerCase());
    const matchesDept = department === "All" || e.department === department;
    return matchesName && matchesDept;
  });

  return (
    <div>
```

```
      {/* ... passes state down, and lets the child raise changes via callbacks */}
      <EmployeeControls
        search={search}
        department={department}
        onSearchChange={setSearch}
        onDepartmentChange={setDepartment}
      />
      <ul>
        {visible.map((e) => (
          <li key={e.id}>
            {e.firstName} {e.lastName} — {e.department}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Part B — Extra Topics (must-know rules that go with state)

B1. The Rules of Hooks (and why)

Hooks (`useState` , and others starting with `use`) have two simple rules:

1. **Only call hooks at the top level** of a component. Never inside an `if` , a loop, or a nested function.
2. **Only call hooks from React function components or your own custom hooks** — never from plain regular functions.

```
function Profile() {
  const [name, setName] = useState(""); // top level - correct

  // WRONG - never do these (shown only as comments so they don't run):
  // if (name) { const [x, setX] = useState(0); } // hook inside an if
  // for (let i = 0; i < 3; i++) { useState(i); }  // hook inside a loop

  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

Why: React does not know your variable names. It remembers each piece of state by the **order** the hooks are called, and that order must be the **same on every render**. An `if` or loop can change the order, so React would hand back the wrong state. Keeping hooks at the top, unconditional, keeps the order stable.

B2. Updating state immutably (arrays & objects)

Never change state in place. Don't `array.push( ... )` or `obj.prop = ...`. Instead build a **brand-new** value and pass it to the setter. React compares the new value to the old one to decide whether to re-render; if you mutate the same object, it may think nothing changed. You already know the tools from Day 1: **spread** `...` , `.map()` , and `.filter()` .

```
function EmployeeListDemo({ newEmployee }: { newEmployee: Employee }) {
  const [employees, setEmployees] = useState<Employee[]>([]);

  // ADD: a new array = old items + one more (never employees.push)
  const add = () => setEmployees([ ... employees, newEmployee]);

  // UPDATE: map builds a new array, replacing only the matching item
  const giveRaise = (id: number) =>
    setEmployees(employees.map((e) => (e.id === id ? { ...e, salary: e.salary + 1000 } :
e)));

  // REMOVE: filter keeps everyone except the matching id
  const remove = (id: number) => setEmployees(employees.filter((e) => e.id !== id));

  return (
    <div>
      <button onClick={add}>Add</button>
      <ul>
        {employees.map((e) => (
          <li key={e.id}>
            {e.firstName} (${e.salary})
            <button onClick={() => giveRaise(e.id)}>Raise</button>
            <button onClick={() => remove(e.id)}>Remove</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

- **Coming from C#:** Think "return a new list/record," similar to using a `record` with `with { ... }` , rather than editing the existing object.

B3. Functional updates — `setCount(c => c + 1)`

You can pass the setter a **function** instead of a value. React calls it with the **latest** state and uses whatever you return. Use this whenever the new value depends on the old value — especially when you update

more than once in a row, or inside an event that may batch updates. It avoids using a "stale" (out-of-date) copy of the state.

```
function TripleCounter() {
  const [count, setCount] = useState(0);

  const addThree = () => {
    setCount((c) => c + 1);
    setCount((c) => c + 1);
    setCount((c) => c + 1); // ends at +3 because each call sees the latest value
  };

  return <button onClick={addThree}>Count: {count}</button>;
}
```

> If you had written `setCount(count + 1)` three times, all three would use the same old `count`, and you'd only move up by 1.

B4. Derived state — compute during render, don't store it

If a value can be **calculated** from state or props, calculate it while rendering — don't put it in its own `useState`. Storing it means you have to remember to keep it in sync, which causes bugs. Filtered lists, counts, and totals are all **derived** values.

```
function EmployeeStats({ employees }: { employees: Employee[] }) {
  // derived: computed fresh during render, NOT stored in state
  const activeCount = employees.filter((e) => e.isActive).length;
  const totalSalary = employees.reduce((sum, e) => sum + e.salary, 0);

  return (
    <p>
      {activeCount} active, total salary ${totalSalary}
    </p>
  );
}
```

> In our app, the **filtered employee list** (`visible` above) is derived state — that's why we compute it during render instead of keeping a separate `useState` for it.

B5. Typing state and events in TypeScript

TypeScript usually **guesses** the type of state from the starting value (`useState(0)` → number). You only pass a type when it can't guess — like an empty array or `null`. For event handlers written as separate functions, type the event so `e.target` is typed for you.

```
function EmployeeForm() {
  // Give a type when React can't guess it (empty array / null):
  // const [employees, setEmployees] = useState<Employee[]>([]);
  // const [selected, setSelected] = useState<Employee | null>(null);

  const [name, setName] = useState(""); // guessed as string
  const [dept, setDept] = useState("All");

  // Type the event; then e.target is fully typed:
  const onName = (e: React.ChangeEvent<HTMLInputElement>) => setName(e.target.value);
  const onPick = (e: React.ChangeEvent<HTMLSelectElement>) => setDept(e.target.value);

  return (
    <div>
      <input value={name} onChange={onName} />
      <select value={dept} onChange={onPick}>
        <option value="All">All</option>
        <option value="IT">IT</option>
      </select>
    </div>
  );
}
```

Handy types to remember: `React.ChangeEvent<HTMLInputElement>` (text/checkbox), `React.ChangeEvent<HTMLSelectElement>` (dropdown), `React.MouseEvent<HTMLButtonElement>` (button click).

B6. KnockoutJS → React state (and DevTools)

The mental jump is small. Here's the map:

KnockoutJS	React
`this.x = ko.observable(0)`	`const [x, setX] = useState(0)`
read: `this.x()`	read: `x` (just the variable)
write: `this.x(5)`	write: `setX(5)`
`ko.observableArray([ ... ])`	`useState<Employee[]>([ ... ])`

KnockoutJS	React
`this.x.subscribe(v => ...)`	React re-renders automatically; use `useEffect` for side effects (Day 5)
computed observable	derived value computed during render (B4)
`data-bind="value: x"`	`value={x} onChange={...}` (controlled input)

Two key differences:

- In Knockout you **called** the observable to read/write it. In React you read the plain variable and **only** write through the setter.
- Knockout re-ran just the affected bindings. React re-runs the whole component function and figures out the minimal DOM change itself — so you think in terms of "given this state, what does the UI look like?"

React DevTools (a free browser extension) is your friend for state, like Knockout's context debugger. Open it, click a component, and the **Components** tab shows its current `state` and `props` live — you can even edit a state value there and watch the UI update. Install it early; it makes learning state much faster.

Quick recap of Day 4

- `useState` gives a component memory; change it only through the setter, and React re-renders.
- Wire `onClick` / `onChange` handlers in JSX to turn user actions into state changes.
- Use plain JS (`? :` , `&&`) for conditional rendering; use `.map()` + a stable `key` for lists.
- **Lift state up** into a parent and pass value + callbacks down — that's how our search + filter app shares data.
- Controlled inputs = `value` + `onChange` (deep forms/validation = Day 6; data fetching with `useEffect` = Day 5).
- Extras: follow the Rules of Hooks, update state **immutably**, use **functional updates** when the new value depends on the old, **derive** don't store, and **type** your state/events. React state ≈ Knockout observables, minus the `()` calls.

2 Detailed Notes — Read Once, Understand Everything

The big idea first (read this twice)

State is a component's own memory.

When that memory changes, React **re-renders** the component — it runs the component function again from top to bottom to rebuild what the screen should look like, and then updates only the parts of the real page that actually changed. Its children re-render too.

You already know this feeling from KnockoutJS. There you write:

```
this.count = ko.observable(0);   // an observable
this.count(5);                   // set it → the UI bindings update by themselves
```

React state is the same idea:

```
const [count, setCount] = useState(0); // like ko.observable(0)
setCount(5);                           // set it → the UI updates by itself
```

The one difference to burn into your memory:

- **KnockoutJS** updates *only the specific DOM bindings* that read that observable.
- **React** re-runs the *entire component function* and rebuilds a lightweight copy of the UI (the "virtual DOM"), compares it to the last one, and patches the real page.

In .NET/WPF terms: it's like `INotifyPropertyChanged` firing `PropertyChanged` so bound controls refresh — except React re-runs the whole "render method" instead of nudging one property. So: **you never touch the DOM by hand, and you never mutate state by hand. You call the setter, and React does the rest.**

---

1. What state is, and why props aren't enough

Props are the values a parent passes *into* a component. They come from outside and are **read-only** — the child must not change them. Think of props like the arguments to a C# method: the caller gives them to you; you don't reassign them to change the caller's world.

But a component often needs to *remember something of its own* that changes over time: is a panel open? what did the user type in the search box? which employees are loaded? That "remembered, changeable" data is **state**.

Here is the classic trap. A plain variable does **not** work, because changing it does not tell React to re-render:

```
function BrokenCounter() {
  let count = 0; // a normal variable — NOT state

  return (
    <button
      onClick={() => {
        count = count + 1;   // the number really does go up in memory...
        console.log(count);  //  ...you can see it in the console...
      }}
    >
      Clicked {count} times {/*  ...but this text NEVER updates on screen */}
    </button>
  );
}
```

The variable changes, but React has no idea it should redraw. That's why we need `useState`.

> **Quick rule:** props = data from outside (read-only). State = data the component owns and can change (which triggers a re-render).

---

2. `useState` — the `[value, setValue]` pair

`useState` is a **hook** (a special function React gives you, whose name starts with `use`). You call it inside your component and it hands you back an array with exactly two things:

1. the **current value**
2. a **setter function** to change it

We destructure them (just like array destructuring from Day 1):

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // 0 = initial value; count is inferred as number

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

- The argument to `useState` (`0` here) is the **initial value** — used only on the **first** render.
- `count` is the value **right now**.
- Calling `setCount( ... )` does two things: it saves the new value **and** tells React "please re-render this component." On the next render, `useState` gives you back the updated value.

Typing it: most of the time TypeScript infers the type from the initial value (`useState(0)` → `number`, `useState("")` → `string`). When the initial value can't describe the full type — like something that starts empty but will hold an object — tell TS explicitly with `useState<T>`:

```
// starts as null, later holds an Employee
const [selected, setSelected] = useState<Employee | null>(null);
```

---

3. Rules of Hooks (just one rule you must not break)

Only call hooks at the top level of your component — never inside an `if`, a loop, or a nested function.

Why? React does not know the *names* of your hooks. It tracks them purely by **call order**: "first `useState` I saw, second `useState` I saw, ..." On every render it must see the *same hooks in the same order*. An `if` around a hook could change that order between renders and quietly corrupt your state.

```
function Example({ loggedIn }: { loggedIn: boolean }) {
  const [name, setName] = useState("");   // OK: top level, runs every render

  // if (loggedIn) {
  //   const [age, setAge] = useState(0);  // WRONG: hook hidden inside an if
  // }

  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

Do the branching *after* the hooks, not around them. (Also: only call hooks from React components or from your own custom hooks — not from plain helper functions.)

---

4. Events — `onClick`, `onChange`, and "pass vs call"

React events look like HTML attributes but are camelCase (`onClick`, `onChange`, `onSubmit`) and you hand them a **function**, not a string.

The single most common beginner bug is **calling** the handler instead of **passing** it:

```
function Toolbar() {
  const save = () => console.log("saved");

  return (
    <div>
      <button onClick={save}>Save</button>            {/* GOOD: pass the function */}
      {/* <button onClick={save()}>Save</button> */}   {/* BAD: this RUNS save() during
render */}
      <button onClick={() => save()}>Save later</button> {/* GOOD: wrap it when you need to
pass arguments */}
    </div>
  );
}
```

- `onClick={save}` — you give React the function; React calls it *when clicked*.
- `onClick={save()}` — you call it *right now, during render*, and hand React whatever it returned. Almost never what you want.
- `onClick={() => save(id)}` — wrap it in an arrow when you need to pass arguments.

Typing the event: when you write a named handler, type its event with React's built-in types. Two you'll use constantly:

```
function SearchInput() {
  const [text, setText] = useState("");

  // e.target.value is the text the user typed
  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setText(e.target.value);
  };

  return <input value={text} onChange={handleChange} />;
}
```

`React.ChangeEvent<HTMLInputElement>` for inputs, `React.MouseEvent<HTMLButtonElement>` for button clicks. When you write the handler inline (`onChange={(e) => ...}`), TypeScript already knows the type of `e` for you — no annotation needed.

---

5. State updates are asynchronous and batched

This surprises everyone coming from C#. After you call `setCount(...)`, **the `count` variable does NOT change on the next line.** The new value only shows up on the *next render*. React also **batches** multiple updates in one event and applies them together.

```
function Stepper() {
  const [count, setCount] = useState(0);

  const addThreeBuggy = () => {
    setCount(count + 1); // count is 0 here → asks for 1
    setCount(count + 1); // count is STILL 0 → asks for 1
    setCount(count + 1); // count is STILL 0 → asks for 1
    // Result: 1, not 3
  };

  const addThreeCorrect = () => {
    setCount((prev) => prev + 1); // prev = latest queued value
    setCount((prev) => prev + 1); // builds on the one before
    setCount((prev) => prev + 1); // Result: 3
  };

  return (
    <>
      <span>{count}</span>
      <button onClick={addThreeBuggy}>+3 (buggy)</button>
      <button onClick={addThreeCorrect}>+3 (correct)</button>
    </>
  );
}
```

The rule: if the new state depends on the old state, use the **functional update** form `setCount(prev => ...)`. React feeds you the very latest value, so stacked updates work correctly. If the new value doesn't depend on the old one (e.g. `setText(e.target.value)`), the plain form is fine.

---

6. Update arrays and objects *immutably* (never mutate)

React decides whether to re-render by checking if the state value is a **different reference** than before (like C# reference equality / `ReferenceEquals`). If you mutate the *same* array or object in place, the reference is unchanged, so React thinks "nothing changed" and skips the update.

So: **never** `push`, `splice`, or assign to a property of state. Instead, make a **new** array/object with the change applied, using spread (`...`), `map`, and `filter` from Day 1:

```
const [employees, setEmployees] = useState<Employee[]>([]);

// ADD — new array with the old items plus the new one
setEmployees((prev) => [...prev, newEmployee]);

// UPDATE one — map returns a new array; only the matching item is replaced
setEmployees((prev) =>
  prev.map((e) => (e.id === 3 ? { ...e, salary: e.salary + 1000 } : e))
);

// REMOVE — filter returns a new array without the matching item
setEmployees((prev) => prev.filter((e) => e.id !== 3));
```

Notice the pattern for update: `{ ...e, salary: e.salary + 1000 }` copies every field of the employee, then overwrites just `salary`. That's the immutable way to "change one property." Same idea for a single object in state: never `obj.x = 1`; do `setObj(prev => ({ ...prev, x: 1 }))`.

---

7. Controlled inputs (`value` + `onChange`)

A **controlled input** means React state is the single source of truth for what's in the box. You wire two things together:

- `value={state}` — the box always *shows* the state.
- `onChange={...}` — every keystroke *updates* the state.

This is exactly a KnockoutJS two-way `value` binding, but written by hand as a one-way-down + one-way-up loop:

```
function SearchBox() {
  const [search, setSearch] = useState("");

  return (
    <div>
      <input
        type="text"
        placeholder="Search employees…"
        value={search}                                    // state → input
        onChange={(e) ⇒ setSearch(e.target.value)}        // input → state
      />
      <p>Searching for: {search}</p>
    </div>
  );
}
```

If you set `value={search}` but forget `onChange`, the box becomes read-only (state never changes, so the displayed value never changes). That loop — value down, change up — is what "controlled" means. (This is enough for a search/filter box. **Full form handling and validation are Day 6.**)

---

8. Conditional rendering with state

Now that a value can change, you can show/hide UI based on it. In JSX you use plain JavaScript expressions:

- `condition && <Thing />` — render `<Thing />` only when the condition is true.
- `condition ? <A /> : <B />` — pick one of two.
- Always handle the **empty state** ("nothing to show") — beginners forget this and get a confusing blank screen.

```
function EmployeePanel({ employees }: { employees: Employee[] }) {
  const [showInactive, setShowInactive] = useState(false);

  const visible = showInactive
    ? employees
    : employees.filter((e) => e.isActive);

  return (
    <div>
      <button onClick={() => setShowInactive((v) => !v)}>
        {showInactive ? "Hide inactive" : "Show inactive"}
      </button>

      {visible.length === 0 ? (
        <p>No employees to show.</p>          // empty state
      ) : (
        <ul>
          {visible.map((e) => (
            <li key={e.id}>{e.firstName} {e.lastName}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

---

9. Dynamic lists + keys, now that the list can change

You render a list with `.map()` (Day 3). What's new on Day 4 is that **state can add and remove items**, so the list changes over time. That makes the `key` prop even more important.

A **key** is a stable id that lets React match "which item is which" between renders, so when you remove #3, React removes *that one row* instead of redrawing everything and mixing up the rows.

```
function EmployeeListDemo() {
  const [employees, setEmployees] = useState<Employee[]>([]);

  const addEmployee = (emp: Employee) =>
    setEmployees((prev) => [...prev, emp]);

  const raiseSalary = (id: number) =>
    setEmployees((prev) =>
      prev.map((e) => (e.id === id ? { ...e, salary: e.salary + 1000 } : e))
    );

  const removeEmployee = (id: number) =>
    setEmployees((prev) => prev.filter((e) => e.id !== id));

  return (
    <>
      <button
        onClick={() =>
          addEmployee({
            id: Date.now(),
            firstName: "New",
            lastName: "Hire",
            department: "Sales",
            salary: 50000,
            isActive: true,
            joinDate: "2026-01-01",
          })
        }
      >
        Add
      </button>

      <ul>
        {employees.map((e) => (
          <li key={e.id}>   {/* stable id - NOT the array index */}
            {e.firstName} {e.lastName} - ${e.salary}
            <button onClick={() => raiseSalary(e.id)}>Raise</button>
            <button onClick={() => removeEmployee(e.id)}>Remove</button>
          </li>
        ))}
      </ul>
```

```
    </>
  );
}
```

Use a stable, unique id (`e.id`) as the key — not the array index. With add/remove, indexes shift around, and index-keys make React reuse the wrong rows (wrong text stuck in an input, wrong item deleted). A real database id is perfect.

---

10. Lifting state up (the day's centerpiece)

Sooner or later two components need to agree on the same data. Example: a search box, a department dropdown, and the list — the list must react to *both* controls.

The rule: when several components need the same state, move that state **up** to their closest common parent. Then pass the data **down** as props, and pass **callbacks down** so children can ask the parent to change it. This is the phrase you'll hear forever: **"data down, events up."**

The parent owns `search` and `department`. The children are simple and controlled by the parent:

```
type SearchBarProps = {
  value: string;
  onChange: (next: string) => void;
};

function SearchBar({ value, onChange }: SearchBarProps) {
  return (
    <input
      type="text"
      placeholder="Search by name…"
      value={value}
      onChange={(e) => onChange(e.target.value)}
    />
  );
}

type DepartmentFilterProps = {
  value: string;
  departments: string[];
  onChange: (next: string) => void;
};

function DepartmentFilter({ value, departments, onChange }: DepartmentFilterProps) {
  return (
    <select value={value} onChange={(e) => onChange(e.target.value)}>
      <option value="All">All departments</option>
      {departments.map((d) => (
        <option key={d} value={d}>{d}</option>
      ))}
    </select>
  );
}

function EmployeeList({ employees }: { employees: Employee[] }) {
  if (employees.length === 0) return <p>No matching employees.</p>;
  return (
    <ul>
      {employees.map((e) => (
        <li key={e.id}>{e.firstName} {e.lastName} — {e.department}</li>
      ))}
    </ul>
```

```
  );
}

function EmployeeDashboard({ allEmployees }: { allEmployees: Employee[] }) {
  // State lives HERE, in the common parent, because two children depend on it.
  const [search, setSearch] = useState("");
  const [department, setDepartment] = useState("All");

  // Derived data — computed during render, not stored (see section 11).
  const departments = Array.from(new Set(allEmployees.map((e) => e.department)));

  const filtered = allEmployees.filter((e) => {
    const fullName = `${e.firstName} ${e.lastName}`.toLowerCase();
    const matchesSearch = fullName.includes(search.toLowerCase());
    const matchesDept = department === "All" || e.department === department;
    return matchesSearch && matchesDept;
  });

  return (
    <div>
      {/* data flows DOWN via props; changes flow UP via callbacks */}
      <SearchBar value={search} onChange={setSearch} />
      <DepartmentFilter
        value={department}
        departments={departments}
        onChange={setDepartment}
      />
      <p>{filtered.length} of {allEmployees.length} shown</p>
      <EmployeeList employees={filtered} />
    </div>
  );
}
```

Notice `onChange={setSearch}` — you can pass the setter straight down as the callback. The children don't own anything; they just display a value and report changes upward. The parent is the single source of truth, so search and filter always stay in sync.

---

11. Derived state vs stored state (compute, don't duplicate)

If a value can be **calculated from state you already have**, calculate it during render — do **not** put it in its own `useState`. Extra state that mirrors other state is the #1 source of "the screen is out of sync" bugs, because now you have to remember to update it everywhere.

```
// DON'T: a second copy you must keep in sync by hand
// const [filtered, setFiltered] = useState<Employee[]>([]);

// DO: just compute it each render from the real state
const filtered = allEmployees.filter(/* ... search + department ... */);
```

In the dashboard above, both `departments` and `filtered` are **derived** — plain `const`s computed from `allEmployees`, `search`, and `department`. Because React re-runs the whole function on every state change, these recompute automatically and are always correct. (Think of it like a KnockoutJS `ko.computed` — but you get it for free just by writing a normal expression.)

Rule of thumb: store the *smallest* set of raw facts in state (what the user typed, what they picked, the data you loaded). Everything else, compute.

One-line summary

State is a component's own memory managed with `useState`; you change it only through the setter (immutably, using functional updates when it depends on the old value), React re-renders to match, and when several components need the same data you lift it to their common parent — data down, events up — while computing anything derivable instead of storing it.

*(Next: **Day 5** adds `useEffect` and loading real data. **Day 6** goes deep on forms + validation.)*

3 Common Mistakes to Avoid

Quick reminder before we start. In React, a piece of **state** is a value that, when it changes, makes the screen re-draw itself. This is almost exactly a **KnockoutJS observable**: in Knockout you write `mySalary(60000)` and the bindings update; in React you call `setSalary(60000)` and the component re-renders. Every mistake below is really just *"how do I tell React the value changed?"* done wrong.

All snippets below assume these two lines at the top of the file, plus our familiar `Employee` shape:

```
import { useState, type ChangeEvent } from "react";

type Employee = {
  id: number;
  firstName: string;
  lastName: string;
  department: string;
  salary: number;
  isActive: boolean;
  joinDate: string;
};
```

---

1. Changing state in place (mutating) instead of making a new copy

*"Mutate" just means **change a thing where it sits**, without replacing it.* React does **not** look inside your array or object. It only checks: *"Is this the same box in memory as before?"* (this is called checking the **reference**). If you push into the same array, it's still the same box, so React thinks nothing changed and the screen does **not** update.

```
const [employees, setEmployees] = useState<Employee[]>([]);

function addEmployee(newEmp: Employee) {
  // ❌ push changes the SAME array — same box, so React re-renders nothing
  employees.push(newEmp);
  setEmployees(employees);
}
```

```
function addEmployee(newEmp: Employee) {
  // ✅ build a BRAND-NEW array — a new box tells React "something changed"
  setEmployees([...employees, newEmp]);
}
```

Same rule for objects — never edit one field in place, spread into a new object:

```
const [emp, setEmp] = useState<Employee>(seedEmployee);

// ❌ emp.salary = 60000; setEmp(emp);   // same object, no re-render
// ✅ new object with just salary swapped:
setEmp({ ...emp, salary: 60000 });
```

> **Knockout / C# view:** In C# think of `record` + `with` (`emp with { Salary = 60000 }` gives you a *new* record). In Knockout, `observableArray.push()` worked because Knockout wrapped the array for you — React does **not** wrap anything, so *you* must hand it a new array/object every time.

---

2. Expecting the value to change *immediately* after you call the setter

Calling `setCount(...)` does **not** change `count` on the very next line. React schedules the update and applies it **before the next render** (it also groups several updates together — this grouping is called **batching**). So the old variable still holds the old value right after you set it.

```
const [count, setCount] = useState(0);

function handleClick() {
  setCount(count + 1);
  console.log(count); // ❌ still prints the OLD value (0), not 1
}
```

If you need the new value, use it directly (`count + 1`), or read it in the next render — don't expect `count` itself to have flipped mid-function.

> **Knockout view:** In Knockout, `count(count() + 1)` updated the observable right away, so people expect the same here. React is different: the *variable* stays put until the next render.

---

3. Forgetting the functional update when the new value depends on the old one

Because the setter reads the value from *this* render, calling it twice in a row uses the **same** old number both times — so you only move by one, not two.

```
function addTwo() {
  // ❌ both lines read the same old count → screen goes up by only 1
  setCount(count + 1);
  setCount(count + 1);
}
```

When the next value is built **from the previous one**, pass a function. React feeds it the very latest value each time.

```
function addTwo() {
  // ✅ each call receives the freshest value → goes up by 2
  setCount((prev) => prev + 1);
  setCount((prev) => prev + 1);
}
```

Rule of thumb: *new value depends on old value → use* `set(prev => ...)`.

---

4. Storing "derived" data in state instead of calculating it during render

Derived data = data you can *work out* from other data (e.g. a filtered list is worked out from `employees` + `search`). If you keep it in its own `useState`, you now have **two sources of truth** that you must keep in sync by hand — and one day you'll forget, and the screen goes **stale** (shows old data).

```
const [employees, setEmployees] = useState<Employee[]>([]);
const [search, setSearch] = useState("");

// ❌ a second copy you must remember to refill every time employees or search change
const [filtered, setFiltered] = useState<Employee[]>([]);
```

Just calculate it while rendering. It's a plain variable — always fresh, no syncing:

```
const [employees, setEmployees] = useState<Employee[]>([]);
const [search, setSearch] = useState("");

// ✅ recomputed on every render from the real state — never out of sync
const filtered = employees.filter((emp) =>
  emp.firstName.toLowerCase().includes(search.toLowerCase())
);
```

> **Knockout view:** This is your old friend `ko.computed` — a value that recalculates itself from other observables. In React, *rendering is the computed*. Only put a value in `useState` if the user (or an event) sets it directly; if you can derive it, derive it.

---

5. Breaking the Rules of Hooks (calling a Hook inside `if` / `for` / a nested function)

React tracks your `useState` calls **by their order**, and that order must be **identical on every render**. So Hooks must always run at the **top level** of the component — never inside a condition, a loop, or a helper function.

```
function EmployeeList({ showList }: { showList: boolean }) {
  if (showList) {
    // ❌ never call a Hook inside if / for / a nested function:
    // const [open, setOpen] = useState(false);
  }
  // ...
}
```

Call the Hook first, then decide *what* to show:

```
function EmployeeList({ showList }: { showList: boolean }) {
  // ✅ Hook runs every render, in the same order
  const [open, setOpen] = useState(false);

  if (!showList) return null; // conditions go AFTER the Hooks

  return (
    <button onClick={() => setOpen(!open)}>
      {open ? "Hide" : "Show"} details
    </button>
  );
}
```

> **Tip:** The ESLint rule `react-hooks/rules-of-hooks` catches this for you — keep it on.

---

6. Using the array index as the `key` for a list that can change

A **key** is a small label React puts on each list item so it can tell them apart between renders (like a primary key / dictionary key in C#). If you use the position number (`index`) as the key, then adding, deleting, or

re-ordering rows shifts everyone's number — React matches up the wrong rows and you get mixed-up or "stuck" UI (e.g. a checkbox stays ticked on the wrong employee).

```
// ❌ index changes when the list reorders/deletes → React confuses rows
{employees.map((emp, index) => (
  <EmployeeCard key={index} employee={emp} />
))}
```

```
// ✅ a stable id that belongs to the row itself
{employees.map((emp) => (
  <EmployeeCard key={emp.id} employee={emp} />
))}
```

Use the index **only** for a list that never changes order and never adds/removes items.

---

7. Writing `onClick={handleClick()}` — that runs it *now*, not on click

`onClick` wants **the function itself**, not the *result* of calling it. Adding `()` runs it immediately while React is rendering (wrong timing — and in TypeScript it's usually a type error too, because the click handler slot doesn't accept the function's return value).

```
// ❌ this CALLS handleDelete during render, not on click (and won't type-check):
// <button onClick={handleDelete(emp.id)}>Delete</button>

// ✅ needs an argument? wrap the call in an arrow so it runs on the click:
<button onClick={() => handleDelete(emp.id)}>Delete</button>

// ✅ no argument? just pass the reference — no parentheses:
<button onClick={handleRefresh}>Refresh</button>
```

> **Knockout view:** Like `click: handleDelete` in a `data-bind` — you name the handler; you don't invoke it yourself.

---

8. A controlled input with `value` but no `onChange`

A **controlled input** means the box's text comes *from state*. If you give it `value` but no `onChange`, React holds the box fixed to that state — typing does nothing (it becomes read-only) and React logs a warning.

```
const [search, setSearch] = useState("");

// ❌ value with no onChange → box is frozen / read-only:
// <input value={search} />

// ✅ value + onChange: state drives the box, and typing updates the state
<input
  value={search}
  onChange={(e) => setSearch(e.target.value)}
/>
```

Prefer to **type your event handler** so `e` is checked (`ChangeEvent<HTMLInputElement>` = "the event you get when an input's value changes"):

```
function handleSearch(e: ChangeEvent<HTMLInputElement>) {
  setSearch(e.target.value);
}
// <input value={search} onChange={handleSearch} />
```

> Deep form handling and validation come on **Day 6** — for now, `value` + `onChange` is all you need for search and filter boxes.

---

9. Copying a prop into state (then the two drift apart)

`useState(someProp)` only reads the prop **once**, on the first render. After that it's a private copy — if the parent later sends a **new** value, your copy quietly ignores it and shows stale data.

```
function EmployeeCard({ employee }: { employee: Employee }) {
  // ❌ snapshot taken once; later changes from the parent are ignored
  const [emp] = useState(employee);
  return <h3>{emp.firstName} {emp.lastName}</h3>;
}
```

```
function EmployeeCard({ employee }: { employee: Employee }) {
  // ✅ just read the prop — it always reflects what the parent sends
  return <h3>{employee.firstName} {employee.lastName}</h3>;
}
```

If a child truly needs to *edit* the value, don't copy it — **lift the state up** (next point) so the parent owns it and passes it down.

10. Cramming everything into one component instead of lifting shared state up

When two parts of the screen need the **same** value (e.g. the search box and the list both care about the search text), that value should live in their **closest shared parent**, not be duplicated in each child. The parent owns the state and passes down the value + a callback. This is called **lifting state up** — one source of truth.

```
// ✅ the PARENT owns the shared state; children get data + callbacks via props
function EmployeeDashboard() {
  const [search, setSearch] = useState("");
  const [employees] = useState<Employee[]>(seedEmployees);

  const visible = employees.filter((emp) =>
    emp.firstName.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <>
      <SearchBox value={search} onSearch={setSearch} />
      <EmployeeList employees={visible} />
    </>
  );
}

function SearchBox({
  value,
  onSearch,
}: {
  value: string;
  onSearch: (text: string) => void;
}) {
  return (
    <input value={value} onChange={(e) => onSearch(e.target.value)} />
  );
}
```

(`EmployeeList` and `seedEmployees` are defined elsewhere in your app.)

> **Knockout view:** Same idea as putting one observable on a shared view model and binding both the search box and the list to it. One value, shared from above — not two separate copies trying to stay in sync.

Quick recap

❌ Don't	✅ Do
`arr.push(x)` / `obj.prop = x` then set	Set a **new** array/object (`[...arr, x]` , `{ ...obj, prop: x }`)
Read the state variable right after setting it	Use the new value directly; it updates next render
`setCount(count + 1)` twice	`setCount(prev => prev + 1)` when it depends on the old value
Keep a `filteredList` in `useState`	**Compute** it during render (like a `ko.computed`)
`useState` inside `if` / `for` / a nested fn	Call every Hook at the **top level**, same order each render
`key={index}` on a changing list	`key={emp.id}` — a stable id
`onClick={handleClick()}`	`onClick={handleClick}` or `onClick={() => handleClick(id)}`
`<input value={x} />` alone	`<input value={x} onChange={...} />`
`useState(propValue)` to mirror a prop	Use the prop directly, or lift state to the parent
One giant component holding everyone's data	**Lift** shared state to the closest common parent

4 Concept Notes — Recall & Interview (copy by pen)

> Copy these by pen. Quick point-by-point recall for Day 4 (state basics), plus the classic React interview answers. Keep it short — one glance per concept.

State & useState

State

Definition:

- State is data a component owns and can change over time; when it changes, React re-renders the component (draws it again with the new data).
- Knockout: state ~ a `ko.observable()` — read it, set it, and the view refreshes by itself.
- C#: like a private field on a class, but changing it auto-updates the screen.

✅ **Advantages:**

- Lets the UI react to clicks and typing without touching the DOM (the live HTML) by hand.

❌ **Disadvantages / watch-outs:**

- Too much state is hard to follow; keep only what really changes.

💡 **Interview tip:**

- State is local and private to the component; props come from the parent.

📝 **Example:**

- `const [count, setCount] = useState(0)` — one piece of state plus its setter.

useState

Definition:

- `useState` is the hook (built-in React helper) that adds one piece of state to a function component and returns `[value, setValue]`.
- Knockout: `useState` ~ `ko.observable()`, but instead of `name()` / `name("Het")` you read `name` and call `setName("Het")`.

✅ **Advantages:**

- Simple; call it many times for many independent values.
- Type it in TS: `useState<string>("")` so the value has a known type.

❌ **Disadvantages / watch-outs:**

- The setter replaces the value — it does not merge fields like a C# object update. You rebuild objects/arrays yourself.

💡 **Interview tip:**

- Calling the setter schedules a re-render; it does not change the variable on the current line.

📝 **Example:**

- `const [name, setName] = useState("")` — array destructuring gives value + setter.

State vs props

Definition:

- Props are read-only inputs passed in from the parent; state is data the component owns and can change itself.
- C#: props ~ method/constructor parameters (read-only inputs); state ~ the object's own fields.

✅ **Advantages:**

- Clear ownership: parent controls props, component controls its own state.

❌ **Disadvantages / watch-outs:**

- Never write to props inside a child; if a child must change something, the parent owns the state and passes a callback.

💡 **Interview tip:**

- Props flow down and are immutable inside the child; state is local and mutable via its setter. The same value can be state in the parent and a prop in the child.

📝 **Example:**

- `<EmployeeCard employee={emp} />` — `employee` is a prop here; `emp` may be state in the parent.

Derived state (compute, don't store)

Definition:

- Derived state is any value you can calculate from existing state or props — so calculate it during render, don't give it its own `useState`.
- Knockout: derived state ~ `ko.computed()` — it recalculates from other observables instead of holding its own value.

✅ **Advantages:**

- One source of truth; the computed value can never go stale (out of date).

❌ **Disadvantages / watch-outs:**

- Storing a filtered list in state means you must remember to update it on every change — easy to forget, easy bug.

💡 **Interview tip:**

- If you can compute it from other state, don't put it in state.

📝 **Example:**

- `const activeCount = employees.filter(e => e.isActive).length` — computed each render, never stored.

Hooks & Events

Hooks & the Rules of Hooks

Definition:

- Hooks are special functions (names start with `use`) that let function components use React features like state; the Rules of Hooks say WHERE you may call them.
- Rule 1: call hooks only at the top level — never inside `if`, loops, or nested functions.
- Rule 2: call hooks only from React function components or custom hooks — not from plain JS functions.

✅ **Advantages:**

- The same call order every render lets React match each hook to its stored value.

❌ **Disadvantages / watch-outs:**

- A hook inside an `if` changes the call order and breaks React — the most common hooks bug.

💡 **Interview tip:**

- "Why can't hooks be conditional?" — React tracks hooks by call order, so the order must be identical on every render.

📝 **Example:**

- `// wrong: useState() inside an if` — move `useState` above the `if`.

Event handling & synthetic events

Definition:

- You handle events by passing a function to props like `onClick` / `onChange`; React wraps the native browser event in a cross-browser `SyntheticEvent` (React's own event object).
- C#: handlers ~ events/delegates (`button.Click += ...`), but attached inline in JSX.

✅ **Advantages:**

- Same event API in every browser; you attach handlers right in the JSX.

❌ **Disadvantages / watch-outs:**

- Pass the function, don't call it: `onClick={handleClick}`, not `onClick={handleClick()}`.
- Type the event in TS, e.g. `React.ChangeEvent<HTMLInputElement>`.

💡 **Interview tip:**

- A SyntheticEvent is React's wrapper over the native event that gives one consistent API across browsers.

📝 **Example:**

- `onChange={e => setSearch(e.target.value)}` — read the typed text from the event.

Updating State the Right Way

Immutable state updates

Definition:

- Never change state in place; make a NEW object/array and pass it to the setter.
- C#: like returning a new `record` with `with { }` instead of editing fields.

✅ **Advantages:**

- React compares references (object identity) to decide re-renders; a new reference reliably triggers an update.

❌ **Disadvantages / watch-outs:**

- Mutating (`arr.push( ... )` , `obj.x = 1`) may not re-render and causes hard-to-find bugs.

💡 **Interview tip:**

- "Why must state be immutable?" — React detects change by reference identity, so a mutated same-reference object looks unchanged and may skip the re-render.

📝 **Example:**

- `setList([ ... list, newItem])` — spread makes a new array instead of `push` .
- `setEmp({ ... emp, salary: 100 })` — spread copies, then overrides one field.

Why state updates are async / batched

Definition:

- Calling a setter does not update the value immediately; React schedules the change and batches several updates into one re-render.

✅ **Advantages:**

- Batching means fewer re-renders and better performance.

❌ **Disadvantages / watch-outs:**

- Reading the state variable right after calling its setter still gives the OLD value.

💡 **Interview tip:**

- "Is setState sync or async?" — treat it as async; the new value shows up on the next render, not the next line.

📝 **Example:**

- `setCount(count + 1); console.log(count)` — logs the OLD count.

Functional updates

Definition:

- Pass a function to the setter (`setX(prev => ...)`) when the new value depends on the previous one.

✅ **Advantages:**

- Always uses the latest value, so several updates in a row don't overwrite each other.

❌ **Disadvantages / watch-outs:**

- Calling `setCount(count + 1)` three times may only add 1; use the function form for reliable increments.

💡 **Interview tip:**

- Use the updater function whenever the next state is computed from the current state.

📝 **Example:**

- `setCount(prev => prev + 1)` — safe increment based on the latest value.

Inputs & Data Flow

Controlled vs uncontrolled components (brief)

Definition:

- A controlled input gets its value from state (`value={x}` + `onChange`); an uncontrolled input keeps its own value in the DOM and you read it later (e.g. via a ref).
- Knockout: controlled input ~ `value` bound to an observable — but in React you wire the update yourself in `onChange` .

✅ **Advantages:**

- Controlled = state is the single source of truth; easy to filter, reset, or check.

❌ **Disadvantages / watch-outs:**

- `value` without `onChange` makes a read-only input (React warns). Deep form handling + validation is Day 6.

💡 **Interview tip:**

- Controlled = React state drives the value; uncontrolled = the DOM holds the value.

📝 **Example:**

- `<input value={search} onChange={e => setSearch(e.target.value)} />` — controlled search box.

Lifting state up

Definition:

- When two components need the same data, move that state up to their closest common parent and pass it down as props.

✅ **Advantages:**

- One shared source of truth; siblings stay in sync.

❌ **Disadvantages / watch-outs:**

- Lifting too high causes "prop drilling" (passing props through many layers); Context or a state library fixes that later.

💡 **Interview tip:**

- "What is lifting state up?" — moving shared state to a common ancestor so multiple children read/update the same value.

📝 **Example:**

- Search text lives in `<Dashboard>` and is passed to both the search box and the list.

One-way data flow (data down, events up)

Definition:

- Data flows DOWN through props; children report changes UP by calling callback functions the parent passed in.
- C#/Knockout: callback props ~ events/delegates the child raises and the parent handles.

✅ **Advantages:**

- Easy to trace: you always know where data comes from and who changes it.

❌ **Disadvantages / watch-outs:**

- A child can't change parent state directly — it must call the callback prop.

💡 **Interview tip:**

- Data flows one way (parent → child); children communicate up via callbacks.

📝 **Example:**

- `<EmployeeCard onDelete={() ⇒ removeEmp(emp.id)} />` — child calls up, parent owns the change.

Keys in dynamic lists (reinforced)

Definition:

- When you render a list with `.map()`, give each item a stable, unique `key` so React can track which item changed, moved, or was removed.

✅ **Advantages:**

- Correct and efficient re-renders; keeps each row's input state matched to the right row.

❌ **Disadvantages / watch-outs:**

- Don't use the array index as key if the list can reorder/filter/delete — it causes wrong rows and stale UI.

💡 **Interview tip:**

- "Why keys?" — they give list items a stable identity so React updates only the minimum needed.

📝 **Example:**

- `employees.map(e => <EmployeeCard key={e.id} ... />)` — stable id as key, not the index.